

Portfolio Watcher

Design- en Keuzesdocument (geïmplementeerd)

Datum: 16 januari 2026 | **Scope:** Keuzes die op dit moment in de applicatie zijn doorgevoerd.

Opmerking: deze tekst is geschreven als verantwoording richting de leeruitkomsten (met focus op LO1-LO4). Inhoud is gebaseerd op mijn design/keuzes-aantekeningen en feedbackmomenten tijdens het bouwen.

Inhoud (Word kan hier automatisch een inhoudsopgave van maken via References > Table of Contents).

1. Context en doel

Portfolio Watcher is een ASP.NET Razor Pages applicatie waarmee een gebruiker portfolios en trades kan registreren en bekijken. De kern van het semester zit voor mij in het aantoonbaar toepassen van OO-modelleren, lagenarchitectuur, SOLID (met nadruk op S, O, D) en testbaarheid. Daarom zijn veel keuzes gemaakt met onderhoudbaarheid en uitbreidbaarheid als primaire kwaliteitsattributen.

2. Architectuurkeuze: drie lagen (GUI - Domein - Data)

Ik hanteer een klassieke 3-laagse architectuur: GUI (Razor Pages) -> Domein (businesslogica/use cases) -> Data (database toegang). Ik heb bewust geen aparte applicatielaag toegevoegd, omdat dit project qua schaal vooral baat heeft bij helderheid en directe traceerbaarheid tussen use case, domeinservice en repository. Een extra laag zou in mijn context vooral extra mappen/boilerplate toevoegen zonder nieuwe verantwoordelijkheid.

- GUI-laag: PageModels + ViewModels voor user input/validatie en het presenteren van data.
- Domeinlaag: domeinmodellen (Trade, Portfolio, Symbol, User) + services die use cases uitvoeren.
- Data-laag: repositories met SQL, mapping naar DTOs en database-connection utilities.

3. Afhankelijkheden en testbaarheid (Dependency Inversion)

Een belangrijk doel was het loskoppelen van de domeinlaag van concrete database-implementaties. Daarom staan repository-interfaces in de domeinlaag en implementaties in de data-laag. De domeinservices werken tegen abstracties (interfaces) en krijgen implementaties via dependency injection. Dit maakt de domeinlogica unit-testbaar zonder echte database (mock/fake repositories).

- Domein definieert contracten (bijv. ITradeRepository) en verwacht gedrag, geen concrete Sql-classes.

- Data implementeert deze contracten (bijv. `SqlTradeRepository`) en bevat de SQL/connection details.
- De composition root (`Program.cs` / DI container) bepaalt welke implementatie wordt geïnjecteerd.

4. DTOs vs domeinmodellen (scheiding opslag en business)

Ik gebruik DTOs als data-carriers voor opslag/transport, omdat het datamodel niet 1-op-1 gelijk is aan het domeinmodel. De `TradeDTO` bevat alleen velden die direct in de database staan (`Id`, `SymbolId`, `BuyPrice`, `SellPrice`, `Shares`, `PortfolioId`). Het domeinmodel `Trade` bevat daarnaast afgeleide waarden zoals `PositionSize`, `ProfitLoss` en `ChangePercentage`. Deze berekeningen horen bij het domein (businessbetekenis) en blijven daarom uit de DTOs.

4.1 Mapping-keuze

In enkele onderdelen map ik ook wanneer het technisch gezien niet strikt nodig is (bijv. repository zou direct domeinmodel kunnen vullen). Ik kies hier bewust voor consistentie: elke laag spreekt zijn eigen modellen en de vertaling zit op vaste plekken. Dit voorkomt dat database-details (zoals column names/nullable regels) langzaam doorsijpelen naar de domeinlaag.

5. Encapsulation en validatie

Mijn domeinmodellen bewaken hun eigen geldige state. Fields zijn private en mutatie verloopt via constructors/methodes met validatie. ViewModels hebben public setters (vereist voor model-binding), maar de domeinlaag vertrouwt GUI-input nooit blind en valideert opnieuw.

- Gebruik van constructors om minimale vereisten af te dwingen (bijv. `prijs > 0`, `shares > 0`).
- Gebruik van private setters/public getters om ongecontroleerde mutatie te vermijden.
- Gebruik van Range-validatie in plaats van `Required` voor value types om aanwezigheid + businessgrenzen af te dwingen.

6. Rich vs anemic: bewust een hybride focus

Voor dit project heb ik vooral een rich domain benadering gebruikt voor `Trade`: berekeningen die direct bij een trade horen (zoals P/L) zitten in het `Trade`-model zelf. CRUD-achtige orchestratie (opslaan, ophalen, sorteren) zit in services. Dit houdt de businessbetekenis dicht bij de data waar het over gaat, zonder dat services anemische models moeten “oplappen”.

7. Uitbreidbaarheid: sorteren/zoeken/filteren via strategie-interfaces

Voor sorteer-/zoek-/filterfunctionaliteit heb ik een interface (strategie) tussen de caller en de concrete implementatie gezet. Daardoor kan ik nieuwe varianten toevoegen (bijv. sorteren op winst, op datum, op symbool) zonder bestaande code te wijzigen, behalve op de plek waar ik een keuze maak voor een concrete strategie.

- `ISortStrategy`/`ITradeSort` als contract voor sorteer-gedrag.
- Concrete classes per sorteerwijze.
- GUI kiest een strategie en roept dezelfde service-methode aan.

8. Datamodel-keuzes (ERD)

Ik gebruik een aparte TradeId als primary key, omdat een trade een zelfstandige entiteit is. PortfolioId en SymbolId zijn foreign keys. Een samengestelde primary key (PortfolioId + SymbolId) zou de use case blokkeren waarin meerdere trades met hetzelfde symbool in hetzelfde portfolio bestaan.

8.1 Many-to-many Portfolio <-> Symbol zonder extra koppeltabel

Conceptueel is er een many-to-many relatie tussen Portfolio en Symbol. In mijn domein zijn de symbolen van een portfolio echter volledig afgeleid van de trades binnen dat portfolio. De Trade-tabel fungeert daarom impliciet als koppeltabel: elke trade heeft een portfolio en een symbool. Een extra koppel-tabel zou geen extra businesswaarde opslaan en daarom vermijd ik die complexiteit.

9. Belangrijke implementatiebevinding: decimal separator en validatie

Ik liep tegen een opslagprobleem aan waarbij prijzen als integers in de database terechtkwamen (bijv. 12.43 werd 1243). De oorzaak bleek een cultuur/locale mismatch: de user-input moest komma-gescheiden zijn, terwijl .NET client-side validatie (DataAnnotations) punten verwachtte. Ik heb dit opgelost door client-side validatie te overriden zodat komma-invoer geaccepteerd wordt, waarna opslag correct ging. Dit is een belangrijk leerpunt: validatie is niet alleen een UX-keuze maar kan ook direct data-integriteit beïnvloeden.

10. Initialisatie en dependency injection in de GUI

In een eerste versie instantieerde ik repositories/services direct in de PageModels met new(). Dat werkte, maar leidde tot duplicatie, slechtere testbaarheid en rommelige controllers. Ik heb dit omgezet naar het registreren van dependencies in de DI container (Program.cs) zodat Razor Pages de benodigde services kan injecteren. Dit maakt de GUI dunner en beter onderhoudbaar.

11. Versiebeheer en werkwijze

Ik werk met GitHub branches (main/development) en kleine commits met duidelijke messages. Per use case maak ik bij voorkeur een aparte branch. Bij merge conflicten heb ik de conflicten handmatig opgelost en bewust gekozen voor de werkende implementatie uit development, gevolgd door een merge commit.

12. Belangrijke designcorrectie: Trade -> Portfolio relatie

Ik had in het domeinmodel eerst gekozen om een Trade een Portfolio-object te geven (Trade "heeft" Portfolio), met als argument dat dit invalid states voorkomt. Na feedback heb ik ingezien dat dit niet de beste modellering is: de database dwingt al af dat elke trade bij een portfolio hoort, en in de applicatie-flow is het logischer dat een Portfolio een collectie Trades beheert. De extra referentie in Trade voegt dan vooral coupling toe. Deze keuze is een belangrijk leerpunt over ownership/compositie en over "waar hoort verantwoordelijkheid thuis".

13. SOLID samengevat (bewijsrichting)

In dit project toon ik SOLID vooral aan via S, O en D:

- S (Single Responsibility): services per use case, repositories alleen data-toegang, sorteerklassen alleen sorteren.
- O (Open/Closed): nieuwe sorteer-/filtervarianten toevoegen via nieuwe implementaties van dezelfde interface.
- D (Dependency Inversion): domeinlaag werkt tegen interfaces; implementaties worden geïnjecteerd zodat unit tests mocks kunnen gebruiken.

Bijlage A: gekozen businessregels (samenvatting)

- Open trade wordt afgeleid uit `SellPrice == null` (1 bron van waarheid).
- Trades zijn niet direct aan users gekoppeld; ownership loopt via Portfolio -> User om dubbele relaties te voorkomen.
- Symbolen hebben geen eigenaar; een symbool kan in meerdere portfolios voorkomen.